

Submission Worksheet

Submission Data

Course: IT114-003-F2025

Assignment: IT114 Milestone 1

Student: Alanor S. (ass89)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 11/6/2025 1:10:40 PM

Updated: 11/8/2025 6:26:09 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-1/grading/ass89>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-milestone-1/view/ass89>

Instructions

- Overview Link: <https://youtu.be/9dZPFwi76ak>

1. Refer to Milestone1 of any of these docs:
 2. [Rock Paper Scissors](#)
 3. [Basic Battleship](#)
 4. [Hangman / Word guess](#)
 5. [Trivia](#)
 6. [Go Fish](#)
 7. [Pictionary / Drawing](#)
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
 1. `git checkout Milestone1` (ensure proper starting branch)
 2. `git pull origin Milestone1` (ensure history is up to date)
5. Copy Part5 and rename the copy as Project (this new folder should be in the root of your repo)
6. Organize the files into their respective packages Client, Common, Server, Exceptions
 1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson
7. Fill out the below worksheet
 1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.
8. Once finished, click "Submit and Export"
9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone1`
 4. On Github merge the pull request from Milestone1 to main
10. Upload the same PDF to Canvas
11. Sync Local

- Briefly explain how the server-side waits for and accepts/handles connections

Your Response:

On the server side, the program waits for and handles client connections using a `ServerSocket`. When the server starts, it creates a `ServerSocket` bound to a specified port, the default in this case, and enters a continuous loop while it's running. Inside this loop, it calls `serverSocket.accept()`, a blocking operation; meaning the program pauses and waits until a client attempts to connect. Once a client connects, the server accepts the connection and creates a new `ServerThread` instance to handle communication with that client independently. The server then starts this thread, allowing multiple clients to connect and communicate concurrently without blocking new connections. When a `ServerThread` is initialized, the server assigns it a unique ID and adds it to the default room, in our case the Lobby, so it can interact with other clients. On high level explanation; the server continuously listens for new clients, manages active connections, and handles them with multithreading.



Saved: 11/8/2025 7:21:13 AM

Section #2: (1 pt.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections

```
089-1114-883 (Milestone1)
$ java Project.Server.Server 3001
Server Starting
Server: Listening on port 3001
Room[Lobby]: Created
Server: Created new Room lobby
Server: Waiting for next client
Server: Client connected
Thread[-1]: ServerThread created
Server: Waiting for next client
Thread[-1]: Thread starting
Thread[-1]: Received from my client: Payload[CLIENT_CONNECT] Client Id [0] Message: [null] ClientName: [A1]
Thread[1]: Sending to client: Payload[CLIENT_ID] Client Id [1] Message: [null] ClientName: [A1]
Thread[-1]: Thread starting
Thread[-1]: Received from my client: Payload[CLIENT_CONNECT] Client Id [0] Message: [null] ClientName: [A1]
Thread[1]: Sending to client: Payload[CLIENT_ID] Client Id [1] Message: [null] ClientName: [A1]
Server: "A101 initialized"
```

Show the terminal output of the server receiving multiple connections

Default Being "I obby")

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of rooms being created, joined, and removed (server-side)
- Show the relevant snippets of code that handle room management (create, join, leave, remove) (server-side)

This screenshot shows the VS Code editor with three files open. The leftmost file is a JavaScript file containing code for room management, including functions for creating, joining, and removing rooms. The middle and rightmost files are terminal windows showing the output of the code. The terminal output includes messages such as "Room created", "Room joined", and "Room removed", along with the names of the rooms and the users who interacted with them.

Show the terminal output of rooms being created, joined, and removed (server-side)

This screenshot shows the VS Code editor with three files open. The leftmost file is a JavaScript file containing code for room management, including functions for creating, joining, and removing rooms. The middle and rightmost files are terminal windows showing the output of the code. The terminal output includes messages such as "Room created", "Room joined", and "Room removed", along with the names of the rooms and the users who interacted with them.

Show the terminal output of rooms being created, joined, and removed (server-side)

This screenshot shows the VS Code editor with three files open. The leftmost file is a JavaScript file containing code for room management, including functions for creating, joining, and removing rooms. The middle and rightmost files are terminal windows showing the output of the code. The terminal output includes messages such as "Room created", "Room joined", and "Room removed", along with the names of the rooms and the users who interacted with them.

Show the terminal output of rooms being created, joined, and removed (server-side)

This screenshot shows the VS Code editor with three files open. The leftmost file is a JavaScript file containing code for room management, including functions for creating, joining, and removing rooms. The middle and rightmost files are terminal windows showing the output of the code. The terminal output includes messages such as "Room created", "Room joined", and "Room removed", along with the names of the rooms and the users who interacted with them.

Creating and joining a room

Creating and joining a room

User joining and leaving a room

code for creating a room in Room.java

```

108 public class Server {
109     // Stores name, unique name of the room
110     + @param name If it was created and false if it wasn't
111     + @return DuplicateRoomException
112     //
113     protected void createRoom(String name) throws DuplicateRoomException {
114         final String message = "Room has been created";
115         if (!room.containsKey(name)) {
116             // Room doesn't exist yet
117             throw new DuplicateRoomException(String.format("Room %s already exists", name));
118         }
119         // Room name is now unique (name);
120         // Room set (newName, room);
121         info(String.format("Created new Room %s", name));
122         // * 122-123 protected void unwatchRoom(String name) throws DuplicateRoomEx...
123     }
124     /**
125      * Attempts to serve a client (RunnableThread) between rooms.
126      *
127      * @param name the target room to join
128      * @param client the client being
129      * @throws SocketTimeoutException
130      */
131     protected void doRoom(RunnableThread client) throws SocketTimeoutException {

```

code for creating a room in Server.java

```

151 public class Room implements AutoCloseable {
152     public void handleCreateRoom(ServerThread sender, String roomName) {
153         sender.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format("Room %s already exists", roomName));
154     }
155     // 228-238 public void handleCreateRoom(ServerThread sender, String roomName) {
156     // }
157     public void handleJoinRoom(ServerThread sender, String roomName) {
158         try {
159             ServerThread client = sender.getServerThread();
160             switch (Constants.DEFAULT_CLIENT_ID) {
161                 case CLIENT_CONNECT:
162                     sender.sendMessage(Constants.DEFAULT_CLIENT_ID, String.format("Room %s doesn't exist", roomName));
163             }
164             // 245-251 public void handleJoinRoom(ServerThread sender, String roomName) {
165             // }
166             protected synchronized void handleDisconnect(DatagramSocket sender) {
167                 handleDisconnect((ServerThread) sender);
168             }
169             // 267
170             // expose access to the disconnect action
171             // 269
172             // 270 serverThread
173 }

```

code for joining a room in Room.java

```

143 public class Room {
144     // 145
146     protected void joinRoom(String name, ServerThread client) throws RoomNotFoundException {
147         final String roomName = name.toLowerCase();
148         if (!rooms.containsKey(roomName)) {
149             throw new RoomNotFoundException(String.format("Room %s wasn't found", name));
150         }
151         Room currentRoom = rooms.get(roomName);
152         if (currentRoom != null) {
153             info("Handling client from previous room " + currentRoom.getName());
154             currentRoom.removeClient(client);
155             Room next = rooms.get(roomName);
156             next.addClient(client);
157             // 161-171 protected void joinRoom(String name, ServerThread client) throws
158             // 172
159             protected void removeRoom(Room room) {
160                 rooms.remove(room.getName().toLowerCase());
161                 info(String.format("Removed room %s", room.getName()));
162             }
163             // 164
164             // 165
165 }

```

code for joining a room in Server.java

```

202 protected synchronized void handleDisconnect(ServerThread sender) {
203     disconnect(sender);
204 }
205
206 protected synchronized void handleReverseText(ServerThread sender, String text) {
207     StringBuilder sb = new StringBuilder(text);
208     sb.reverse();
209     String rev = sb.toString();
210     relay(sender, rev);
211 }
212 // 213-214 protected synchronized void handleReverseText(ServerThread sender, String text) {
213 }
214
215 protected synchronized void handleMessage(ServerThread sender, String text) {
216     relay(sender, text);
217 }
218 // end handle methods
219 // 221-227 public class Room implements AutoCloseable

```

leaving/disconnecting in room.java part 1

```

148 protected synchronized void disconnect(ServerThread client) {
149     if (!isRunning()) {
150         return;
151     }
152     ServerThread disconnectingServerThread = client.getServerThread();
153     if (disconnectingServerThread != null) {
154         client.getServerThread().removeClient(disconnectingServerThread.getClientId());
155         if (disconnectingServerThread.getClientId() == disconnectingServerThread.getClientId()) {
156             return true;
157         }
158         boolean failedToSend = disconnectingServerThread.sendMessage(Constants.DEFAULT_CLIENT_ID, Constants.DISCONNECT);
159         if (failedToSend) {
160             System.out.println("Failed to disconnect " + disconnectingServerThread.getClientId() + " from list", disconnectingServerThread.getDisplayMsg());
161             // 176-180
162             return failedToSend;
163         }
164         relay(null, disconnectingServerThread.getDisplayMsg() + " disconnected");
165         disconnectingServerThread.disconnect();
166     }
167     // 188-190
168 }

```

leaving/disconnecting in room.java part 2

```

143 public class ServerThread extends BaseServerThread {
144     // 145
146     protected void processPayload(Payload incoming) {
147         switch (incoming.getPayloadType()) {
148             case CLIENT_CONNECT:
149                 clientName = ((ConnectionPayload) incoming).getClientName().trim();
150                 break;
151             case DISCONNECT:
152                 disconnectingServerThread = client.getServerThread();
153                 break;
154             case MSG_RECV:
155                 currentRoom.handleMessage(this, incoming.getMessage());
156                 break;
157             case BPVMSG:
158                 // 161-171

```


leaving/disconnecting in ServerThread.java. ServerThread.java calls this on PayloadType.DISCONNECT

removing/cleaning up a room in Room.java

removing/cleaning up a room in Server.java

/disconnect command

⇒ Part 2:

Your Response:

The server manages rooms using a `ConcurrentHashMap`, where each room name maps to a `Room` object. With this design, the server can handle multiple requests for a room simultaneously.

Room object. When a new room is created, the createRoom() method checks if a room with the same name already exists; if not, it adds the new Room instance to the map. To join a room, the joinRoom() method first removes the client from their current room, if any, and then adds them to the new one using that room's addClient() method. If a client leaves and the room becomes empty, the room can later be removed from the map using removeRoom(), which deletes it by name.

📄 Saved: 11/8/2025 9:07:43 AM

Section #4: (1 pt.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

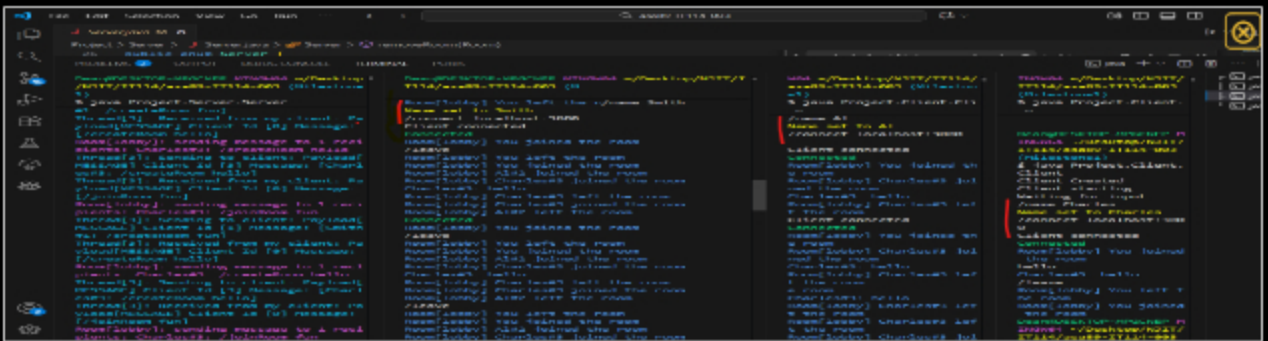
Progress: 100%

📁 Part 1:

Progress: 100%

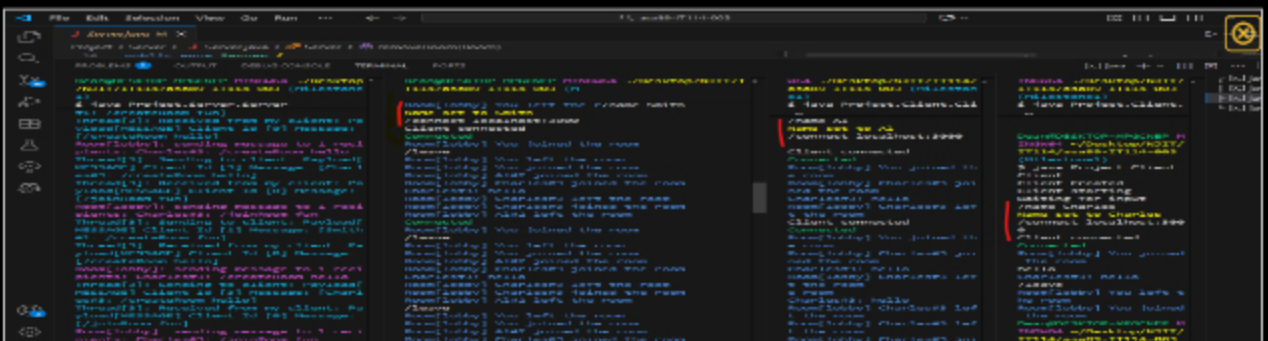
Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected



```
Terminal 1: /name /connect /name /connect /name /connect
Terminal 2: /name /connect /name /connect /name /connect
Terminal 3: /name /connect /name /connect /name /connect
```

Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)



```
Terminal 1: /name /connect /name /connect /name /connect
Terminal 2: /name /connect /name /connect /name /connect
Terminal 3: /name /connect /name /connect /name /connect
```

```

122 private boolean processClientCommand(String cmd) throws IOException {
123     boolean success = false;
124     if (cmd.startsWith("/name ")) {
125         // Handle /name command
126         String name = cmd.substring(7);
127         if (name.length() > 0) {
128             // Update client name
129             // ...
130         } else {
131             // Error: name required
132             // ...
133         }
134     }
135     // ...
136 }

```

Show the relevant snippets of code that handle the processes for /name

```

122 private boolean processClientCommand(String cmd) throws IOException {
123     boolean success = false;
124     if (cmd.startsWith("/name ")) {
125         // Handle /name command
126         String name = cmd.substring(7);
127         if (name.length() > 0) {
128             // Update client name
129             // ...
130         } else {
131             // Error: name required
132             // ...
133         }
134     }
135     // ...
136 }

```

Show the relevant snippets of code that handle the processes for /connect pt 1

```

66 private boolean connect(String address, int port) {
67     try {
68         // Create socket
69         // ...
70     } catch (IOException e) {
71         // Handle exception
72         // ...
73     }
74     return isConnected();
75 }

```

Show the relevant snippets of code that handle the processes for /connect pt 2

```

222 private void processClientData(Payload payload) {
223     // Handle client data
224     // ...
225 }

```

Show the relevant snippets of code that handle the processes for confirmation of being fully setup/conn



Saved: 11/8/2025 7:41:21 AM

Details:

- Briefly explain how the /name and /connect commands work and the code flow that leads to a successful connection for the client

Your Response:

The `/name` command allows the client to set their display name before connecting to the server. When the user types in `/name AI` for example, the input is processed by the `processClientCommand()` method, which detects the `/name` keyword and stores "AI" locally in the `myUser` object without sending anything over the network. The `/connect` command, on the other hand, establishes the actual connection to the server. When the user types `/connect localhost:3000`, the client validates the input, ensures a name has been set, and then calls the `connect()` method to create a socket connection to the specified host and port. Once connected, it sends a `CLIENT_CONNECT` payload containing the client's name to the server. On the server side, the `ServerThread` receives the payload, assigns the name to the client, and responds with a `CLIENT_ID` payload confirming the connection. The client's `listenToServer()` method processes this response, updates the client's ID and name, and prints "Connected" to confirm success. On a high level, `/name` prepares the client's identity, and `/connect` initiates and completes the handshake that establishes the client's connection to the server.



Saved: 11/8/2025 7:41:21 AM

Section #5: (2 pts.) Feature: Client Can Create/j oin Rooms

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

Progress: 100%

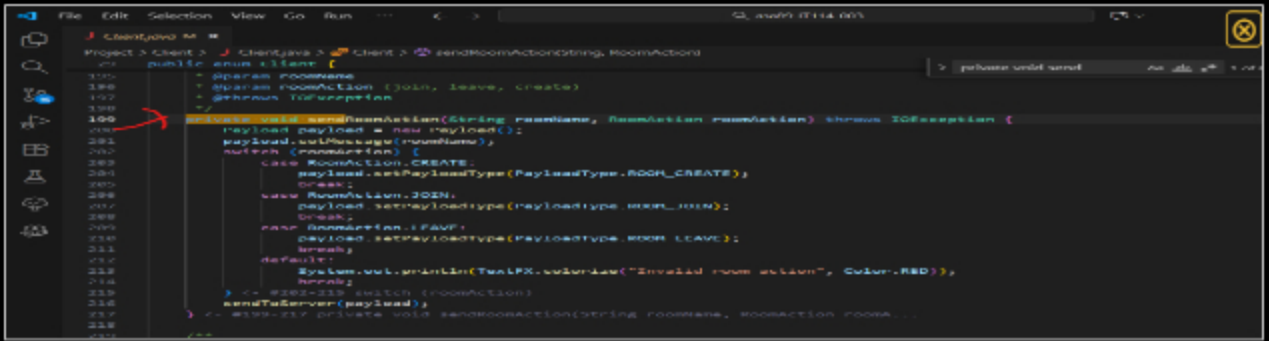
Part 1:

Progress: 100%

Details:

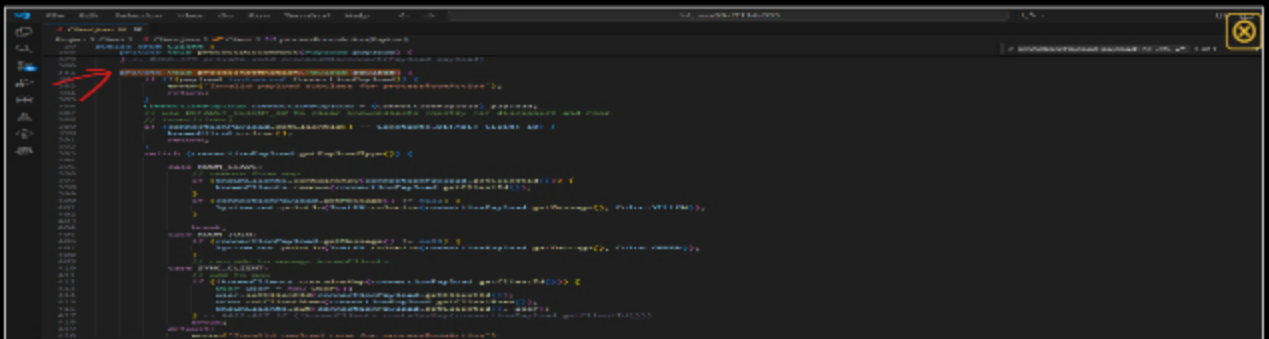
- Show the terminal output of the `/createroom` and `/joinroom`
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining

Show the terminal output of the /createroom and /joinroom Output should show evidence of a successful creation/join in both



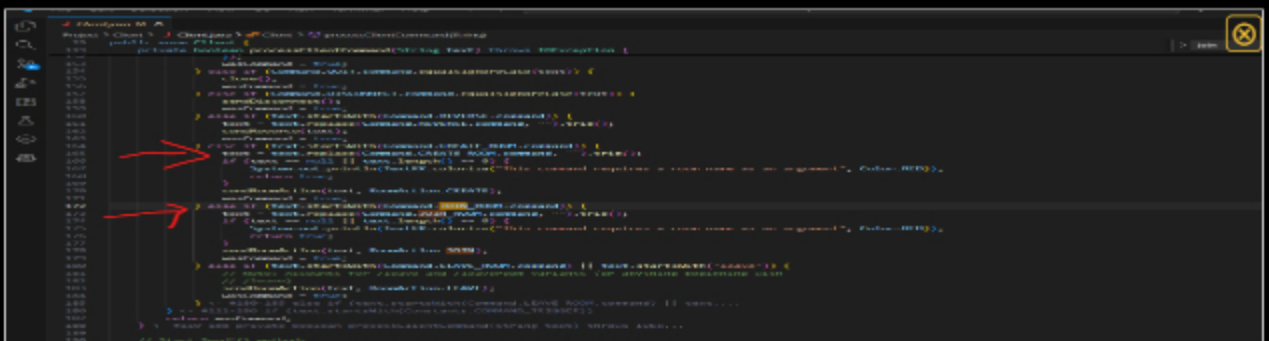
```
1100 public void sendRoomAction(String roomName, RoomAction roomAction) throws IOException {
1101     payload.payload = new payload();
1102     payload.setRoomName(roomName);
1103     switch (roomAction) {
1104         case ROOM_CREATE:
1105             payload.setPayloadType(PayloadType.ROOM_CREATE);
1106             break;
1107         case ROOM_JOIN:
1108             payload.setPayloadType(PayloadType.ROOM_JOIN);
1109             break;
1110         case ROOM_LEAVE:
1111             payload.setPayloadType(PayloadType.ROOM_LEAVE);
1112             break;
1113         default:
1114             System.out.println(TextFX.colorize("Invalid room action", Color.RED));
1115             return;
1116     }
1117     sendToServer(payload);
1118 }
1119 // WS-237 private void sendRoomAction(String roomName, RoomAction roomAction) {
```

Show the relevant snippets of code that handle the client-side processes for room creation and joining



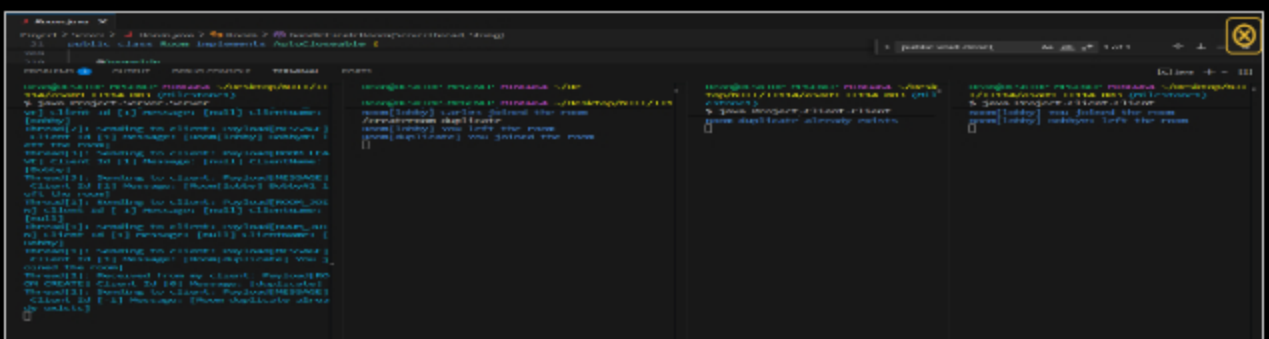
```
1100 public void sendRoomAction(String roomName, RoomAction roomAction) throws IOException {
1101     payload.payload = new payload();
1102     payload.setRoomName(roomName);
1103     switch (roomAction) {
1104         case ROOM_CREATE:
1105             payload.setPayloadType(PayloadType.ROOM_CREATE);
1106             break;
1107         case ROOM_JOIN:
1108             payload.setPayloadType(PayloadType.ROOM_JOIN);
1109             break;
1110         case ROOM_LEAVE:
1111             payload.setPayloadType(PayloadType.ROOM_LEAVE);
1112             break;
1113         default:
1114             System.out.println(TextFX.colorize("Invalid room action", Color.RED));
1115             return;
1116     }
1117     sendToServer(payload);
1118 }
1119 // WS-237 private void sendRoomAction(String roomName, RoomAction roomAction) {
```

Show the relevant snippets of code that handle the client-side processes for room creation and joining



```
1100 public void sendRoomAction(String roomName, RoomAction roomAction) throws IOException {
1101     payload.payload = new payload();
1102     payload.setRoomName(roomName);
1103     switch (roomAction) {
1104         case ROOM_CREATE:
1105             payload.setPayloadType(PayloadType.ROOM_CREATE);
1106             break;
1107         case ROOM_JOIN:
1108             payload.setPayloadType(PayloadType.ROOM_JOIN);
1109             break;
1110         case ROOM_LEAVE:
1111             payload.setPayloadType(PayloadType.ROOM_LEAVE);
1112             break;
1113         default:
1114             System.out.println(TextFX.colorize("Invalid room action", Color.RED));
1115             return;
1116     }
1117     sendToServer(payload);
1118 }
1119 // WS-237 private void sendRoomAction(String roomName, RoomAction roomAction) {
```

Show the relevant snippets of code that handle the client-side processes for room creation and joining



```
1100 public void sendRoomAction(String roomName, RoomAction roomAction) throws IOException {
1101     payload.payload = new payload();
1102     payload.setRoomName(roomName);
1103     switch (roomAction) {
1104         case ROOM_CREATE:
1105             payload.setPayloadType(PayloadType.ROOM_CREATE);
1106             break;
1107         case ROOM_JOIN:
1108             payload.setPayloadType(PayloadType.ROOM_JOIN);
1109             break;
1110         case ROOM_LEAVE:
1111             payload.setPayloadType(PayloadType.ROOM_LEAVE);
1112             break;
1113         default:
1114             System.out.println(TextFX.colorize("Invalid room action", Color.RED));
1115             return;
1116     }
1117     sendToServer(payload);
1118 }
1119 // WS-237 private void sendRoomAction(String roomName, RoomAction roomAction) {
```

duplicate room handled gracefully

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how the `/createroom` and `/join room` commands work and the related code flow for each

Your Response:

When a client issues the `/createroom` or `/joinroom` command, the client-side `Client` class first parses the input and calls the `sendRoomAction()` method. This method creates a `Payload` object, sets its `PayloadType` to either `ROOM_CREATE` or `ROOM_JOIN`, and includes the target room name in the message field. The payload is then sent to the server over the established socket connection. On the server side, the `ServerThread`'s `processPayload()` method receives the payload and routes it to the appropriate handler in the client's current room: `handleCreateRoom()` for creation or `handleJoinRoom()` for joining. These handlers interact with the `Server` to either create a new `Room` object, if the room doesn't already exist, or move the client from their current room to the target room, updating internal room membership structures. The server then sends a `ConnectionPayload` back to the client, confirming the room creation or successful join, which the client processes in `processRoomAction()` to update its local state and display feedback, completing the command's flow.



Saved: 11/8/2025 6:26:09 PM

Section #6: (1 pt.) Feature: Client Can Send Messages

Progress: 100%

≡ Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of a few messages from each of 3 clients
- Include examples of clients grouped into other rooms
- Show the relevant snippets of code that handle the message process from client to server-side and back

```
Thread[5]: Sending to client: Payload[Room] client id [2] Message: [Room] to
attingagain... You left the room
Server: Removed room testingagain...
Room[testingagain...]: closed
Thread[6]: Sending to client: Payload[Room_JOIN] client id [-2] Message: [null]
clientName: [null]
Thread[7]: Sending to client: Payload[Room_JOIN] client id [2] Message: [null]
clientName: [Charles]
Thread[8]: Sending to client: Payload[Room_LEAVE] client id [2] Message: [Room[bi
rthday] you joined the room]
Thread[9]: Received from client: Pay
```

```

Hello
SmithW1: Hello
AlH7: Hello!
AlH7: hows it going
good, and u?
SmithW1: good, and u?
/createroom hiking
Room[123] You left the room
Room[hiking] You joined the room
Room[hiking] Alex joined the room
hows the weather looking for the
hike?
SmithW1: hows the weather lookin
g for the hike
AlH7: still clear

```

```

ALW2: hows it going
Smithw1: good, and u?
Room[124] Smithw1 left the room
/JoinRoom hiking
Room[125] You left the room
Room[hiking] You joined the room
Smithw1: 'hows the weather looking for
the hike
ALW2: still clear
Smithw1: still clear

```

```

$ ./createroom birthday
Room[Testingagain...] You
ou left the room
Room[birthday] You joined
the room
i am the only person here,
just testing
CharlesB: i am the only
person here, just testing

```

```

124a[Room[001]]: Client Id [0] Message: [I
am the only person here, just testing]
Room[birthday]: sending message to 1 re
cipients: Charles#3: I am the only pers
on here, just testing
Thread[5]: Sending to client: Payload[M
ESSAGE] Client Id [0] Message: [Charles
#3: I am the only person here, just tes
ting]

```

Include examples of clients grouped into other rooms AI and Smith in same room (hiking), Charles in room by himself (birthday)

```

Thread[5]: Sending to client: Payload[M
ESSAGE] Client Id [0] Message: [Room[te
stingagain...]: You left the room]
Server: Removed room testingagain...
Room[testingagain...]: closed
Thread[2]: Sending to client: Payload[M
ESSAGE] Client Id [-1] Message: [null]
ClientName: [null]
Thread[2]: Sending to client: Payload[M
ESSAGE] Client Id [3] Message: [null]
ClientName: [Charles]
Thread[2]: sending to client: Payload[M
ESSAGE] Client Id [3] Message: [Room[bi
rthday] You joined the room]
Thread[2]: Received from my client: Pay
load[MESSAGE] Client Id [0] Message: [I
am the only person here, just testing]
Room[birthday]: sending message to 1 re
cipients: Charles#3: I am the only pers
on here, just testing
Thread[2]: sending to client: Payload[M
ESSAGE] Client Id [0] Message: [Charles
#3: I am the only person here, just tes
ting]

```

Show the terminal output of a few messages from each of 3 clients

```

File Edit Selection View Go Run ...
Client.java M
Project > Client > Client.java > Client
287 public enum Client {
288     = @param message
289     = throws IOException
290     // Sends the client's name to the server (what the user desires to be called)
291     = @param name
292     = throws IOException
293     private void sendClientName(String name) throws IOException {
294         ConnectionPayload payload = new ConnectionPayload();
295         payload.setPayloadType(PayloadType.CLIENT_CONNECT);
296         sendToServer(payload);
297     }
298     // Sends the client's name to the server (what the user desires to be called)
299     = @param name
300     = throws IOException
301     private void sendClientName(String name) throws IOException {
302         ConnectionPayload payload = new ConnectionPayload();
303         payload.setPayloadType(PayloadType.CLIENT_CONNECT);
304         sendToServer(payload);
305     }
306 }

```

Client-side: Sending a message to the server

```

File Edit Selection View Go Run ...
Client.java M
Project > Client > Client.java > Client > sendToServer(Payload)
292 public enum Client {
293     // Sends the client's name to the server (what the user desires to be called)
294     = @param name
295     = throws IOException
296     private void sendClientName(String name) throws IOException {
297         ConnectionPayload payload = new ConnectionPayload();
298         payload.setPayloadType(PayloadType.CLIENT_CONNECT);
299         sendToServer(payload);
300     }
301     // Sends the client's name to the server (what the user desires to be called)
302     = @param name
303     = throws IOException
304     private void sendClientName(String name) throws IOException {
305         ConnectionPayload payload = new ConnectionPayload();
306         payload.setPayloadType(PayloadType.CLIENT_CONNECT);
307         sendToServer(payload);
308     }
309     // Sends the client's name to the server (what the user desires to be called)
310     = @param name
311     = throws IOException
312     private void sendClientName(String name) throws IOException {
313         ConnectionPayload payload = new ConnectionPayload();
314         payload.setPayloadType(PayloadType.CLIENT_CONNECT);
315         sendToServer(payload);
316     }
317 }

```

Client-side: Sending a message to the server

```

File Edit Selection View Go Run ...
Client.java M
Project > Client > Client.java > Client > listenToInput()
437 public enum Client {
438     // Listens for keyboard input from the user
439     //
440     //
441     private void listenToInput() {
442         try {
443             Scanner si = new Scanner(System.in);
444             System.out.println("Waiting for input"); // moved here to avoid console spam
445             while (si.hasNextLine()) {
446                 String userInput = si.nextLine();
447                 if (!isEmptyClientCommand(userInput)) {
448                     sendToServer(userInput);
449                 }
450             }
451         } catch (IOException ioe) {
452             System.out.println("Error in listenToInput()");
453             ioe.printStackTrace();
454         }
455         System.out.println("listenToInput thread stopped");
456     }
457 }

```

Triggering message send: From user input

```

File Edit Selection View Go Run ...
Client.java M
Project > Client > Client.java > Client > listenToInput()
437 public enum Client {
438     // Listens for keyboard input from the user
439     //
440     //
441     private void listenToInput() {
442         try {
443             Scanner si = new Scanner(System.in);
444             System.out.println("Waiting for input"); // moved here to avoid console spam
445             while (si.hasNextLine()) {
446                 String userInput = si.nextLine();
447                 if (!isEmptyClientCommand(userInput)) {
448                     sendToServer(userInput);
449                 }
450             }
451         } catch (IOException ioe) {
452             System.out.println("Error in listenToInput()");
453             ioe.printStackTrace();
454         }
455         System.out.println("listenToInput thread stopped");
456     }
457 }

```


Room relays the message

```
serverThread.java
137 public class ServerThread extends BaseServerThread {
138     protected boolean sendClientId() {
139         payload.setClientId(clientId); // Set the client ID in the payload object
140         return sendToClient(payload);
141     } // 117-124 protected boolean sendClientId()
142
143     /**
144      * Sends a message to the client
145      *
146      * @param clientId who it's from
147      * @param message
148      * @return true for successful send
149      */
150     protected boolean sendMessage(long clientId, String message) {
151         Payload payload = new Payload();
152         payload.setPayloadType(PayloadType.MESSAGE);
153         payload.setMessage(message);
154         payload.setClientId(clientId);
155         return sendToClient(payload);
156     } // 117-124 protected boolean sendMessage(long clientId, String message)
157
158     // End Send() Methods
159     @Override
160     protected void processPayload(Payload payload) {
161         // Process the payload
162     }
163 }
```

ServerThread sends back to clients

 Saved: 11/8/2025 8:11:15 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

When a client types a regular message (not a command), the client-side Client class detects that it's not a recognized command in processClientCommand(). It then creates a Payload object with PayloadType.MESSAGE and the message text, and sends it to the server via sendToServer(). On the server side, the ServerThread handling that client receives the payload in processPayload(), recognizes it as a MESSAGE type, and forwards it to the current room's handleMessage() method. The Room object then iterates over all connected clients in that room and relays the message to each client's ServerThread using sendMessage(). Each receiving client processes the incoming payload in its processPayload() method, specifically in processMessage(), and prints the message to the console. Meaning, that a message typed by one client gets delivered to all other clients in the same room in real time.

 Saved: 11/8/2025 8:11:15 AM

Section #7: (1 pt.) Feature: Disconnection

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occurring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process

```

// ClientThread.java
// This class represents a client that connects to a server and sends/receives data.
// It implements the Runnable interface.

import java.io.*;
import java.net.*;
import java.util.*;

public class ClientThread implements Runnable {
    private Socket socket;
    private DataInputStream in;
    private DataOutputStream out;
    private String message;

    public ClientThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            in = new DataInputStream(socket.getInputStream());
            out = new DataOutputStream(socket.getOutputStream());

            // Receive message from server
            message = in.readLine();

            // Send message to server
            out.writeLine(message);

            // Disconnect from server
            disconnect();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void disconnect() {
        try {
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Show examples of clients disconnecting (server should still be active)

```

// ServerThread.java
// This class represents a server that listens for client connections and handles them.
// It implements the Runnable interface.

import java.io.*;
import java.net.*;
import java.util.*;

public class ServerThread implements Runnable {
    private ServerSocket serverSocket;
    private Socket socket;
    private DataInputStream in;
    private DataOutputStream out;
    private String message;

    public ServerThread(ServerSocket serverSocket) {
        this.serverSocket = serverSocket;
    }

    public void run() {
        try {
            socket = serverSocket.accept();
            in = new DataInputStream(socket.getInputStream());
            out = new DataOutputStream(socket.getOutputStream());

            // Receive message from client
            message = in.readLine();

            // Send message to client
            out.writeLine(message);

            // Disconnect from client
            disconnect();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void disconnect() {
        try {
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Show examples of server disconnecting (clients should be active but disconnected)

```

// ClientThread.java
// This class represents a client that connects to a server and sends/receives data.
// It implements the Runnable interface.

import java.io.*;
import java.net.*;
import java.util.*;

public class ClientThread implements Runnable {
    private Socket socket;
    private DataInputStream in;
    private DataOutputStream out;
    private String message;

    public ClientThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            in = new DataInputStream(socket.getInputStream());
            out = new DataOutputStream(socket.getOutputStream());

            // Receive message from server
            message = in.readLine();

            // Send message to server
            out.writeLine(message);

            // Disconnect from server
            disconnect();

            // Reconnect to server
            connect();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void disconnect() {
        try {
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void connect() {
        try {
            socket = new Socket("localhost", 8080);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Show examples of clients reconnecting when a server is brought back online

```

// ServerThread.java
// This class represents a server that listens for client connections and handles them.
// It implements the Runnable interface.

import java.io.*;
import java.net.*;
import java.util.*;

public class ServerThread implements Runnable {
    private ServerSocket serverSocket;
    private Socket socket;
    private DataInputStream in;
    private DataOutputStream out;
    private String message;

    public ServerThread(ServerSocket serverSocket) {
        this.serverSocket = serverSocket;
    }

    public void run() {
        try {
            socket = serverSocket.accept();
            in = new DataInputStream(socket.getInputStream());
            out = new DataOutputStream(socket.getOutputStream());

            // Receive message from client
            message = in.readLine();

            // Send message to client
            out.writeLine(message);

            // Disconnect from client
            disconnect();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void disconnect() {
        try {
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Show the relevant snippets of code that handle the client-side disconnection process

```

// ServerThread.java
// ...
public class ServerThread implements Runnable {
    // ...
    public void run() {
        while (true) {
            // ...
            // Accept a new connection
            Socket s = serverSocket.accept();
            // ...
            // Create a new thread to handle the connection
            ServerThread clientThread = new ServerThread(s);
            // ...
            // Start the thread
            clientThread.start();
        }
    }
}

```

Show the relevant snippets of code that handle the client-side disconnection process

```

// ServerThread.java
// ...
public class ServerThread implements Runnable {
    // ...
    public void run() {
        while (true) {
            // ...
            // Accept a new connection
            Socket s = serverSocket.accept();
            // ...
            // Create a new thread to handle the connection
            ServerThread clientThread = new ServerThread(s);
            // ...
            // Start the thread
            clientThread.start();
        }
    }
}

```

Show the relevant snippets of code that handle the client-side disconnection process

```

// ServerThread.java
// ...
public class ServerThread implements Runnable {
    // ...
    public void run() {
        while (true) {
            // ...
            // Accept a new connection
            Socket s = serverSocket.accept();
            // ...
            // Create a new thread to handle the connection
            ServerThread clientThread = new ServerThread(s);
            // ...
            // Start the thread
            clientThread.start();
        }
    }
}

```

Show the relevant snippets of code that handle the client-side disconnection process

```

// ServerThread.java
// ...
public class ServerThread implements Runnable {
    // ...
    public void run() {
        while (true) {
            // ...
            // Accept a new connection
            Socket s = serverSocket.accept();
            // ...
            // Create a new thread to handle the connection
            ServerThread clientThread = new ServerThread(s);
            // ...
            // Start the thread
            clientThread.start();
        }
    }
}

```

Show the relevant snippets of code that handle the server-side termination process

```

// ServerThread.java
// ...
public class ServerThread implements Runnable {
    // ...
    public void run() {
        while (true) {
            // ...
            // Accept a new connection
            Socket s = serverSocket.accept();
            // ...
            // Create a new thread to handle the connection
            ServerThread clientThread = new ServerThread(s);
            // ...
            // Start the thread
            clientThread.start();
        }
    }
}

```

```
1 // Disconnect from the room and close the connection
2 public void disconnect() {
3     // Set isRunning to false to stop the listening threads
4     isRunning = false;
5     // Close the socket and both the input and output streams
6     try {
7         socket.close();
8     } catch (IOException e) {
9         e.printStackTrace();
10    }
11    // Close the input and output streams
12    try {
13        inputStream.close();
14        outputStream.close();
15    } catch (IOException e) {
16        e.printStackTrace();
17    }
18    // Notify the server that the client has disconnected
19    sendPayload(DISCONNECT, null);
20 }
```

Show the relevant snippets of code that handle the server-side termination process

```
1 // Handle the server-side termination process
2 public void handleDisconnect() {
3     // Get the client's ID from the payload
4     String clientId = payload.getString("clientId");
5     // Remove the client from the room's active users
6     currentRoom.removeUser(clientId);
7     // Update the room's membership count
8     currentRoom.setUsersCount(currentRoom.getUsersCount() - 1);
9     // Broadcast the updated room state to all clients in the room
10    broadcastPayload(ROOM_UPDATE, currentRoom.getState());
11    // Log the disconnection
12    log.info("Client " + clientId + " disconnected from the room.");
13 }
```

Show the relevant snippets of code that handle the server-side termination process



Saved: 11/8/2025 6:24:06 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

On the client side, when a user issues a disconnect command, like /disconnect or /quit, or the client application is closing, the Client class sets isRunning to false and calls close() or closeServerConnection(). This closes the socket and both the input and output streams to the server, meaning no further data is sent or received. Any listening threads, such as listenToInput() and listenToServer(), naturally exit their loops because isRunning is false or the streams are closed. On the server side, when a client disconnects, either intentionally or due to a network issue, the ServerThread receives a DISCONNECT payload and calls currentRoom.handleDisconnect(), which removes the client from the room and updates the server's room membership structures. The server also takes the client out of its list of active users to free up memory and resources. If the server itself is shutting down, a special cleanup process automatically runs to safely disconnect all connected clients and clear out all rooms using disconnectAll() before the server stops.



Saved: 11/8/2025 6:24:06 PM

Section #8: (1 pt.) Misc

Progress: 100%

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages



Saved: 11/6/2025 2:58:39 PM

Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waketime



waketime



Saved: 11/6/2025 3:00:29 PM

Task #4 (0.25 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how a server can handle multiple clients at the same time using multithreading. By creating a separate thread for each connected client, the server can listen for new connections while also managing messages and commands from existing clients. This means that clients can interact with the server simultaneously without blocking each other.



Saved: 11/8/2025 8:22:50 AM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I think the most straightforward part of this was understanding the logic behind sending and receiving simple messages between the client and server. Once the basic socket connection was established, creating a Payload for a message and printing it on the other side is straightforward compared to managing rooms, handling commands, or making sure there was proper multithreading and disconnect logic. Basically; the client sends, the server receives, the server relays, the client displays.



Saved: 11/8/2025 8:26:06 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was managing rooms and moving clients between them while making sure everything runs smoothly with multiple threads. For example, when a client sends a createRoom or /joinRoom command, the Client class sends a

Client sends a createRoom or joinRoom command, the Client class sends a Payload to the server, and the ServerThread calls methods like `currentRoom.handleCreateRoom()` or `currentRoom.handleJoinRoom()`. These methods update the Server's rooms map and add or remove clients from rooms. Since many clients can be connected at the same time, the server uses a `ConcurrentHashMap` to safely manage rooms without crashing when multiple threads try to change the data at once.



Saved: 11/8/2025 8:31:09 AM